Lógica Difusa con Matlab, SciLab y Arduino: Prototipo para la detección del estado ambiental - Mecanismo de avance autoajustable
Informe de las experiencias

Agustín Encina - Claudio Mousquere - Eduardo Bennesch - Lucas Kucuk - Ruben Syniuk

Introducción

En el presente informe se desarrollan dos experiencias, realizadas para poner en práctica la creación de controladores difusos para determinar, en base a variables obtenidas una respuesta determinada, en el primero, se tienen en cuenta 2 variables meteorológicas, temperatura y humedad lo que permite determinar como salida el "estado del tiempo" y en la otra experiencia, se toma en cuenta la distancia a un objeto en movimiento y calculando la velocidad, se obtiene como salida parámetros que permiten, por medio del hardware utilizado, "perseguir al objeto de referencia".

Experiencia 1: Prototipo detección del estado ambiental

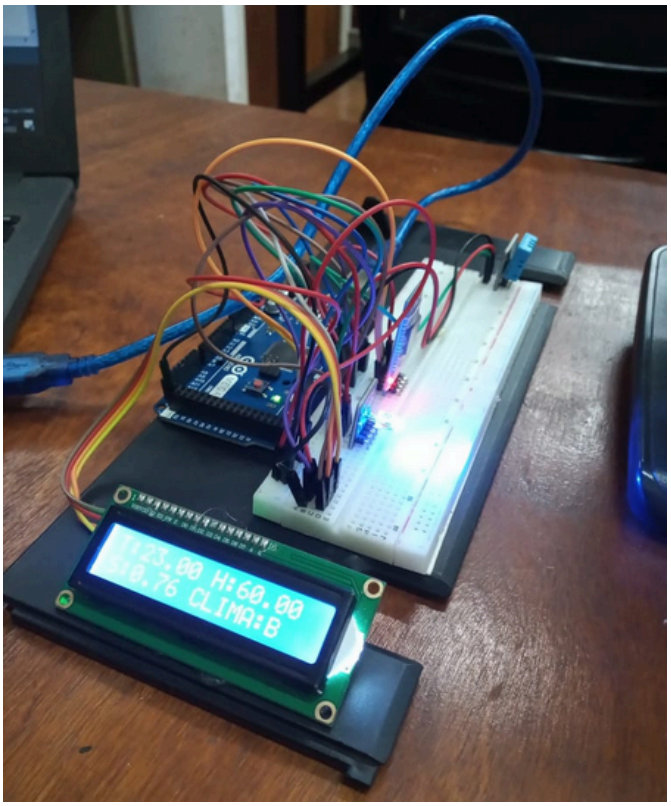

Figura 1 *- Prototipo de detección del estado ambiental.*

El prototipo (**Figura 1**) se programó con un controlador difuso, que obtiene como variables de entradas temperatura y humedad del sensor DHT11 y como salida emite una etiqueta (malo, normal o bueno) según el estado del clima que se puede visualizar en un display lcd.

Además, un led RGB KY016, emite una luz de color haciendo coresponder el rojo para la etiqueta malo, el verde para la etiqueta normal y azul para un clima etiquetado como bueno, cabe destacar que el color del led está relacionado al resultado de la defuzzificación de la función, por lo que emite colores intermedios,

dentro de la gama de los anteriormente mencionados, de acuerdo a que tan “bueno” o “malo” se determine el clima.

Método de Inferencia

Podemos definir como inferencia difusa, a la interpretación de reglas del tipo SI-ENTONCES (IF-THEN) con el fin de obtener las conclusiones de las variables lingüísticas de salida, a partir de los valores de las variables de entrada. La inferencia se basa en el paradigma “Modus Ponens Generalizado” el cual se puede interpretar como la transformación de los grados de cumplimiento del lado izquierdo de las reglas a grados de posibilidad de los lados derechos de las mismas.

Inferencia de Takagi-Sugeno-Kang (TSK)

Es un método basado en reglas difusas donde el consecuente da como resultado una serie de ecuaciones lineales. Este modelo es útil para sistemas complejos de dimensiones mayores a los que se pueden resolver por el método de Mamdani [1].

Inferencia de Mamdani

Este método es más simple y más costoso, computacionalmente hablando, que el TSK, a su vez es el más utilizado [1], dada la naturaleza simple de las experiencias seleccionadas para el presente trabajo se aplicará este método para ambos casos.
El proceso se realiza en los siguientes pasos:

- Fuzzificación de las variables de entrada.
- Evaluación de las reglas.
- Agregación de las salidas de las reglas.
- Defuzzificación.

Fuzzificación

Este proceso permite asociar un valor numérico a un conjunto difuso, asignándole un grado de pertenencia según una etiqueta lingüística a partir de una función de pertenencia [1], a continuación se detallan los pasos realizados en la experiencia.

Funciones de Pertenencia

Para la realización del modelo del controlador difuso de esta experiencia se utilizó el *Matlab Fuzzy Logic Designer*, el cual nos permitió probar distintas funciones de pertenencia, inclusive como se puede observar en la **Figura 2**, utilizar funciones de pertenencia asimétricas. Una de las características de esta herramienta del Matlab, es justamente que permite exportar en un archivo (extensión .fis) este tipo de funciones resolviendo las complejidades matemáticas que puede suponer hacerlas manualmente ya que funciones de pertenencia idénticas y simétricas son mucho más sencillas de programar.

Respecto de la forma seleccionada para las funciones de pertenencia, no se encontraron diferencias apreciables en los pruebas realizadas entre las diferentes posibilidades (triangulares, trapezoidales, gaussianas), por lo que se optó por la forma triangular por ser de más fácil implementación.

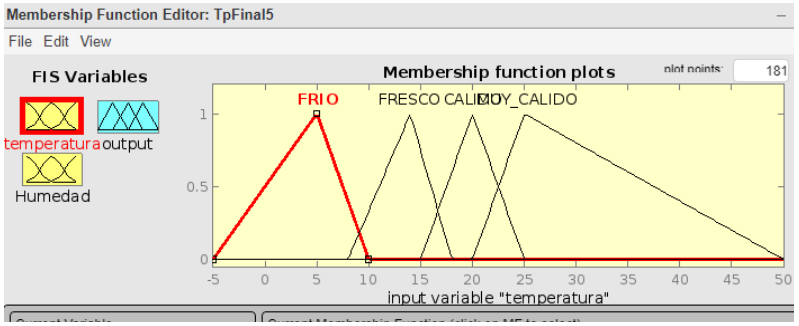

Figura 2 - *Funciones de pertenencia y Etiquetas de la variable Temperatura.*

Etiquetas Lingüísticas

Las etiquetas seleccionadas para esta experiencia fueron las siguientes, para las variables de entrada:

- *Temperatura:* frío, fresco, cálido y muy cálido
- *Humedad:* no confortable (-) , aceptable, agradable, no agradable (+); mientras que para la salida las etiquetas seleccionadas fueron tres:
- *Clima:* malo, normal, bueno.

Cabe destacar que las etiquetas utilizadas y los conjuntos difusos elegidos, en un principio estuvieron basados en trabajos anteriores [2][3], pero posteriormente fueron modificados arbitrariamente con fines experimentales y de acuerdo a criterios personales del grupo.

Todas las funciones definidas, así como sus etiquetas están en forma de imágenes en el **Anexo.**

Conjuntos difusos utilizados

Los conjuntos difusos utilizados para la creación del controlador (**Figura A1**) en la experiencia 1, se determinaron de la siguiente manera:

Conjuntos para la variable de temperatura

- Temperatura (C)
    - frío {-5, 10}
    - fresco {8, 18}
    - cálido {15, 25}
    - muy cálido {20, 50}

Los conjuntos se pueden ver de modo gráfico en la **Figura A2.**

Conjuntos para la variable de humedad

- Humedad (%)
    - no confortable (-) {0, 40}
    - aceptable {35, 70}
    - agradable {50, 60}
    - no confortable (+) {65, 100}

Los conjuntos se pueden ver de modo gráfico en la **Figura A3.**

Conjuntos para el cálculo de salida

- Clima (ESTADO)
    - malo {0, 0.4}
    - normal {0.3, 0.7}
    - bueno {0.5, 1}

Los conjuntos se pueden ver de modo gráfico en la **Figura A4.**

Reglas definidas

La definición de las reglas para el controlador listadas en la **Figura 3** fueron elegidas arbitrariamente y son producto de las pruebas realizadas en el *Matlab Fuzzy Logic Designer.*
Cada una de las reglas determinadas utilizan el operador *and* para su construcción, para evaluar este operador se utiliza habitualmente la T-Norma estándar del mínimo[1] por lo que para la experiencia mantuvimos ese criterio.

1. If (temperatura is FRIO) and (Humedad is No_Confortable-) then (output is MALO) (1)
2. If (temperatura is FRIO) and (Humedad is Aceptable) then (output is NORMAL) (1)
3. If (temperatura is FRIO) and (Humedad is Agradable) then (output is NORMAL) (1)
4. If (temperatura is FRIO) and (Humedad is No_Confortable+) then (output is MALO) (1)
5. If (temperatura is FRESCO) and (Humedad is No_Confortable-) then (output is MALO) (1)
6. If (temperatura is FRESCO) and (Humedad is Aceptable) then (output is NORMAL) (1)
7. If (temperatura is FRESCO) and (Humedad is Agradable) then (output is BUENO) (1)
8. If (temperatura is FRESCO) and (Humedad is No_Confortable+) then (output is MALO) (1)
9. If (temperatura is CALIDO) and (Humedad is No_Confortable-) then (output is MALO) (1)
10. If (temperatura is CALIDO) and (Humedad is Aceptable) then (output is BUENO) (1)
11. If (temperatura is CALIDO) and (Humedad is Agradable) then (output is BUENO) (1)
12. If (temperatura is CALIDO) and (Humedad is No_Confortable+) then (output is MALO) (1)
13. If (temperatura is MUY_CALIDO) and (Humedad is No_Confortable-) then (output is MALO) (1)
14. If (temperatura is MUY_CALIDO) and (Humedad is Aceptable) then (output is BUENO) (1)
15. If (temperatura is MUY_CALIDO) and (Humedad is Agradable) then (output is BUENO) (1)
16. If (temperatura is MUY_CALIDO) and (Humedad is No_Confortable+) then (output is MALO) (1)

Figura 3 *- Reglas del controlador difuso.*

Defuzzificación

Proceso inverso a la fuzzificación, permite asociar a un conjunto difuso un valor numérico y se lleva a cabo para calcular el valor de salida de los modelos difusos[1].

Existen diferentes métodos de defuzzificación y arrojan resultados distintos. El más común y ampliamente utilizado es el del centroide. Para la presente experiencia se mantuvo este criterio usando dicho método.

El método del centroide asocia el centro del área formada por el número difuso, su expresión matemática es la siguiente:

$$c = \frac{\int_S x.\mu(x).dx}{\int_S \mu(x).dx}$$

Con $\mu(x)$ = función de pertenencia del conjunto de salida y $S =$ rango de integración.

Hardware a utilizar

Open Source Mega 2560 R3

La placa es un microcontrolador basado en el ATMega2560-16AU. Tiene 54 pines de entrada / salida digital (de los cuales 14 se pueden usar como salidas PWM), 16 entradas analógicas, 4 UART (hardware de serie estándar), un oscilador de cristal de 16MHz, una conexión USB, un conector de alimentación, ICSP y un botón de reinicio.

Modulo sensor de humedad y temperatura DHT11

Se puede detectar la humedad y la temperatura del entorno. Utiliza el sensor DHT11. Rango de medición de humedad: 20% 95% (rango de 0 grados a 50 grados), error de medición de humedad: +- 5%. Rango de medición de temperatura: 0-50 grados error de medición de temperatura: +- 2 grados. Voltaje de trabajo de 3.3V a 5V.

Módulo Bluetooth HC-06 RS232

Es un módulo bluetooth esclavo diseñado para la comunicación serial inalámbrica. Es un módulo esclavo, lo que significa que puede recibir datos en serie cuando los datos en serie se envían desde un dispositivo bluetooth maestro (dispositivo capaz de enviar datos en serie a través del aire: teléfonos inteligentes, PC).

Módulo emisor Led tricromatico color KY016

El LED del módulo RGB LED está hecho de un plug-in con toda la gama de colores, por R, G, B. El voltaje de entrada PWM de tres pines se puede ajustar en tres colores primarios (rojo / azul / verde) para lograr un efecto de mezcla a todo color .

Display LCD 1602 con I2C

Display LCD de 16 Caracteres / 2 Líneas con Módulo Serie I2C. Puede mostrar letras, números, caracteres especiales y hasta 8 caracteres creados por el usuario. Backlight led color azul. El bus I2C es un tipo de bus serial inventado por PHILIPS. Es un bus serie de alto rendimiento que tiene una función de reglaje de bus y sincronización de dispositivos de alta o baja velocidad requerida por un sistema host múltiple.

Esquema de conexión

En la **Figura 4** se puede observar el siguiente esquema bajo el cual se conectan los componentes detallados con anterioridad, se utilizó un protoboard de 830 para organizar el empalme de las terminales.
El diseño del esquema se realizó con la herramienta Circuito.io [4].

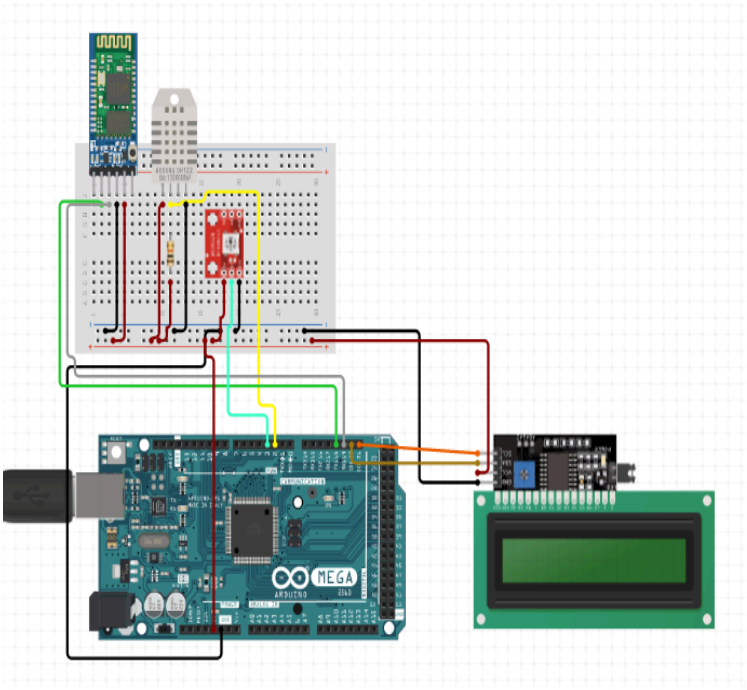

Figura 4 - *Esquema de conexión del Prototipo de Detección de Estado Ambiental.*

Código fuente

Para la programación del controlador difuso, se utilizó un software[1] que convierte el archivo que se exportó del Matlab que tiene una extensión .fis, aquí está configurado el modelo diseñado en el Fuzzy Logic Designer del Matlab, a la extensión .ino, compatible con el IDE Arduino. La descarga e importación de las librerías necesarias para utilizar el hardware seleccionado y la codificación de los componentes de hardware mencionados anteriormente, tanto la entrada como la salida han sido programados en el IDE Arduino. El código se puede ver en la **Tabla A1. En GitHub: https://github.com/rosfix/ld_grp7_deteccion_estado_ambiental**.

Experiencia 2: Mecanismo de Avance Autoajustable

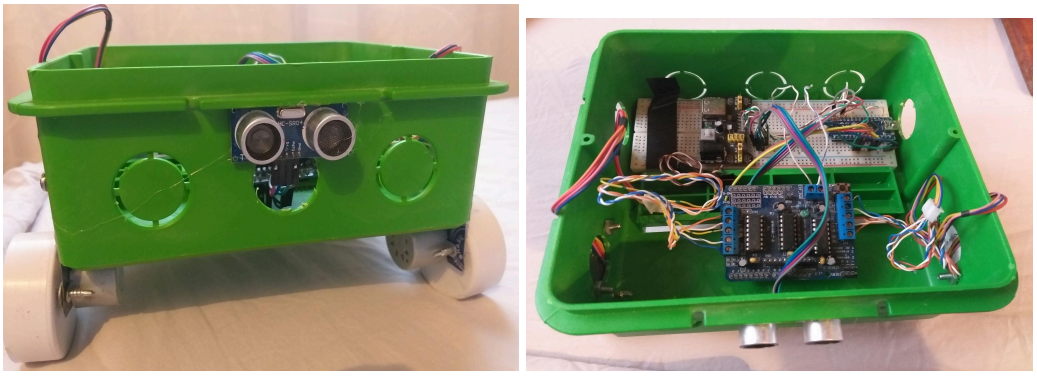

Figura 5: *Prototipo 2 finalizado.*

En esta experiencia (**Figura 5**) se construye un prototipo para desarrollar un controlador difuso que permita realizar un desplazamiento rectilíneo con avance autoajustable de acuerdo a ciertos parámetros espaciales de otro objeto móvil. En esta etapa, el alcance incluye evaluar la distancia y la velocidad del objeto móvil a seguir.

El ajuste automático consiste en determinar el grado de avance para el desplazamiento.

El prototipo realiza la medición de distancia mediante un analizador de eco de ondas sonoras ultrasónicas. La velocidad se calcula por diferencia de mediciones consecutivas.

El avance rectilíneo se efectúa mediante motores de paso a paso.

El software de control se implementa sobre una placa Arduino. Se utilizan las entradas y salidas digitales de manera directa. También se cuenta con el soporte de circuitos auxiliares para el control de los motores.

Método de Control Difuso

Así como en la experiencia anterior, en este caso el modelo de controlador se armó de la siguiente manera

- Método de inferencia: Mamdani
- Método de evaluación del operador *and*: T-Norma estándar del mínimo
- Defuzzificación: método del centroide

Mientras que los conjuntos difusos determinados con sus respectivas etiquetas lingüísticas se detallan en la siguiente sección

Conjuntos difusos utilizados

Los conjuntos difusos utilizados para la creación del controlador (**Figura A5**) en la experiencia 2, se determinaron de la siguiente manera:

[1] http://www.makeproto.com/projects/fuzzy/matlab_arduino_FIST/index.php

Conjuntos para la variable de velocidad

- Velocidad:
  - lento [0 10 20]
  - crucero [9 40 60]
  - corrida [50 70 100]

Los conjuntos se pueden ver de modo gráfico en la **Figura A6.**

Conjuntos para la variable de distancia

- Distancia:
  - cerca [10 30 40]
  - distante [35 70 90]
  - lejos [80 150 200]

Los conjuntos se pueden ver de modo gráfico en la **Figura A7.**

Conjuntos para el cálculo de salida

- Avance
  - Acercar [0 5 10]
  - Seguir [7 20 50]
  - Atrapar [45 70 100]

Los conjuntos se pueden ver de modo gráfico en la **Figura A8.**

Reglas del controlador difuso

En cuanto a las reglas definidas entre las entradas y las salidas, son arbitrarias, todas utilizan el operador *and* y son detalladas en la **Figura 7**, capturada desde el *SciLab,* herramienta utilizada para el modelado de esta experiencia.

```
R1: IF {distancia IS cerca} AND {velocidad IS lento} THEN {avance IS acercar} weigth=1.0
R2: IF {distancia IS distante} AND {velocidad IS crucero} THEN {avance IS seguir} weigth=1.0
R3: IF {distancia IS lejos} AND {velocidad IS crucero} THEN {avance IS atrapar} weigth=1.0
R4: IF {distancia IS cerca} AND {velocidad IS crucero} THEN {avance IS acercar} weigth=1.0
R5: IF {distancia IS cerca} AND {velocidad IS corrida} THEN {avance IS seguir} weigth=1.0
R6: IF {distancia IS distante} AND {velocidad IS lento} THEN {avance IS acercar} weigth=1.0
R7: IF {distancia IS distante} AND {velocidad IS corrida} THEN {avance IS atrapar} weigth=1.0
R8: IF {distancia IS lejos} AND {velocidad IS lento} THEN {avance IS seguir} weigth=1.0
R9: IF {distancia IS lejos} AND {velocidad IS corrida} THEN {avance IS seguir} weigth=1.0
```

Figura 7 - *Captura de reglas del controlador.*

Componentes Hardware

Open Source Nano V3.0

ARDUINO NANO V.3 ATMEGA328P-AU COMPATIBLE CHIP CH340 Tiene más o menos la misma funcionalidad del Arduino Duemilanove, pero en un paquete diferente. Le falta solo un conector de alimentación de CC, y funciona con un cable USB Mini-B en lugar de uno estándar.

Adafruit motor shield

Permite el gobierno de 4 motores de corriente continua más dos servos o dos motores de paso a paso.

Sensor Ultrasonido HC-SR04

El sensor ultrasónico HC-SR04 ofrece un rango de medición de 2cm a los 500cm, con una precisión de aproximadamente 3mm, este módulo ultrasónico funciona de manera similar que sus pares, con la única diferencia que lleva por separado el pin de entrada y el de salida.

Fuente de Alimentación para Protoboard V2 MB-102 3.3V 5V

La Fuente de Alimentación para Protoboard V2 MB-102 3.3V 5V es ideal para proyectos o prototipos en donde se requiera una fuente regulada.

Esquema de conexión

En la **Figura 6** se detalla el esquema de conexión del segundo prototipo propuesto, se utilizó un protoboard de 830 puntos para organizar el empalme de las terminales y una fuente de alimentación 5v para proveer de energía al proyecto. El diseño del esquema se realizó con la herramienta Circuito.io [4]

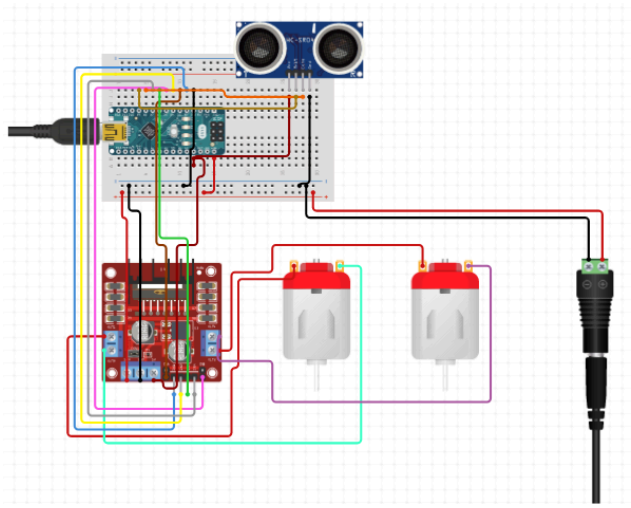

Figura 6 - *Esquema de conexión del Mecanismo de Avance Autoajustable.*

Código fuente

Para el desarrollo del código fuente, se utilizó la librería denominada eFLL (Embedded Fuzzy Logic Library)[2] que se importó al IDE de arduino. A partir de ello se procede a instanciar el objeto para crear los conjuntos difusos, las salidas y la creación de las reglas que fueron diseñadas en sciFLT. El código se puede ver en la **Tabla A2**.

Conclusiones

Existen múltiples herramientas destinadas a la creación e implementación, para proyectos de Lógica Difusa de forma práctica y accesible. Los generadores de código reducen drásticamente los tiempos de programación, por otro lado las interfaces de conexión simplifican las tareas de prototipado.

Los componentes de hardware utilizados en el presente trabajo, son de bajo costo y fáciles de conseguirlos en el mercado local, combinados con otros elementos de elaboración propia, hacen que este tipo de actividades no se transformen en una labor hercúlea; también se han explorado e investigado herramientas

[2] https://www.arduino.cc/reference/en/libraries/efll/

gratuitas y disponible online, que permitieron una rápida diagramación del controlador y modelo a presentar, a estas actividades se les incorpora el uso de las herramientas anteriormente detalladas, este ecosistema de herramientas facilitaron el diseño, desarrollo e implementación de estos prototipos expuestos.

Respecto de los resultados obtenidos para ambas experiencias podemos concluir que se lograron los objetivos planteados desde la cátedra, estos objetivos fueron el diseño, desarrollo, implementación, depuración y puesta en funcionamiento de un controlador difuso embebido.

Referencias

[1] S. D. Sanahuja, "Sistemas de Control con Lógica Difusa: Métodos de Mamdani y de Takagi-Sugeno-Kang (TSK)," UniversitatJaume, p. 73, 2017, [Online]. Available: http://repositori.uji.es/xmlui/handle/10234/173788.

[2] H. Gutierrez, W. Cajo, A. Mauriano, and H. De La Cruz, "Riego Automatizado," no. mm, pp. 4–9, 2014, [Online]. Available: http://v-beta.urp.edu.pe/pdf/id/11239/n/control-difuso-de-una-estacion-meteriologica-en-una-plantacion-con-riego-automatizado.

[3] D. L. Bittner, "FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS CONTROL DIFUSO DE TEMPERATURA Y HUMEDAD DE UN DANIELA LASSO BITTNER PROFESORA GUÍA : DORIS SÁEZ HUEICHAPAN PROFESOR CO-GUÍA : CARLOS MUÑOZ POBLETE COMISIÓN" 2022.

[4] Circuit Design App for Makers- circuito.io. (s. f.). Circuit Design App for Makers- circuito.io. http://www.circuito.io

ANEXO

MATLAB: capturas del controlador difuso - experiencia 1

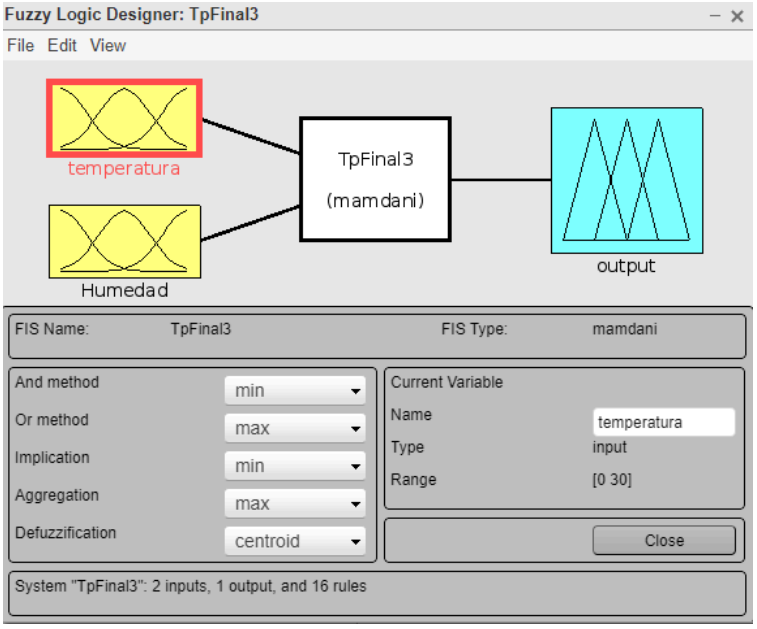

Figura A1 *- Vista previa del esquema para el controlador difuso, experiencia 1.*

Conjuntos difusos para la variable de temperatura

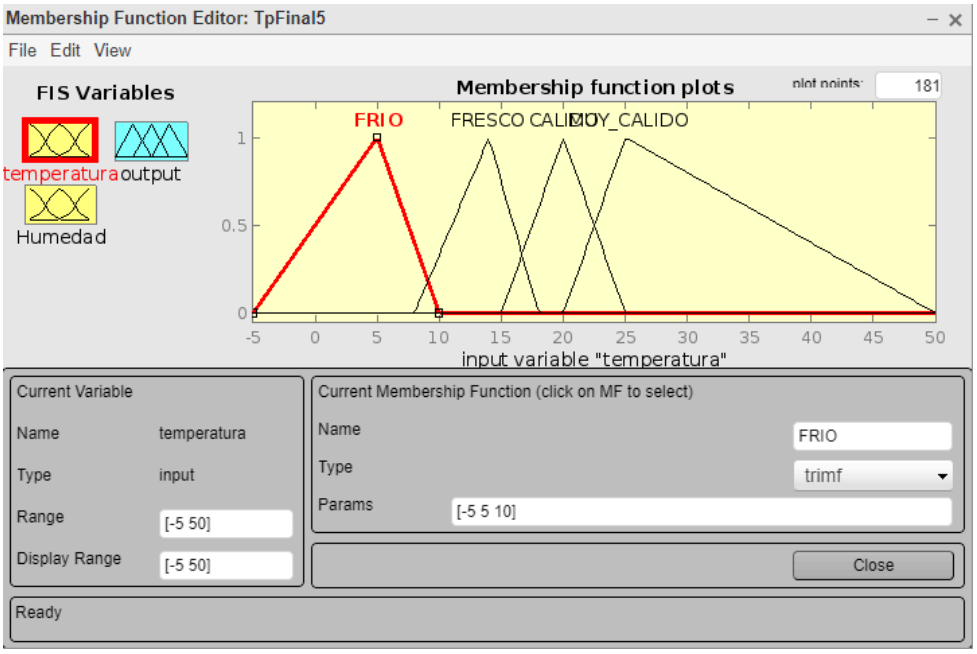

Figura A2 *- Conjuntos difusos, variable de temperatura.*

Conjuntos para la variable de humedad

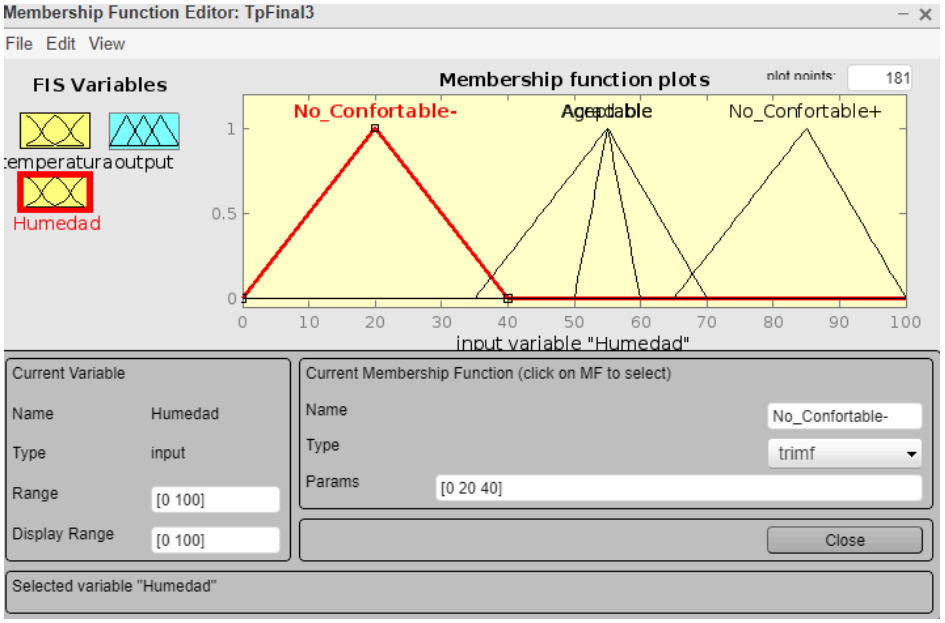

Figura A3 *- Conjuntos difusos, variable de humedad.*

Conjuntos para el cálculo de salida

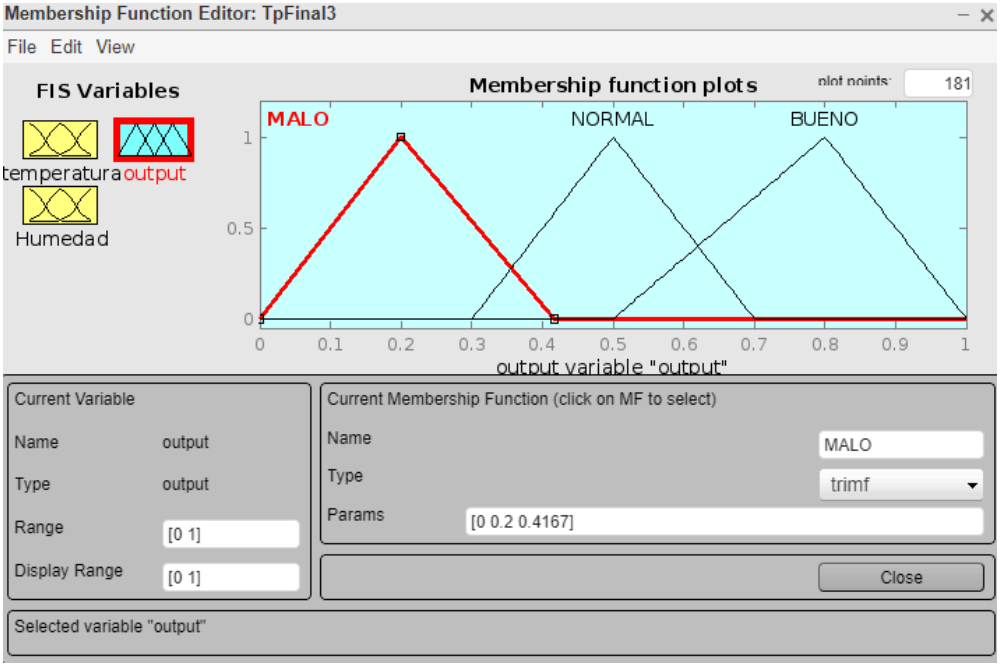

Figura A4 *- Conjuntos difusos, variable de salida.*

sciFLT: capturas del controlador difuso - experiencia 2

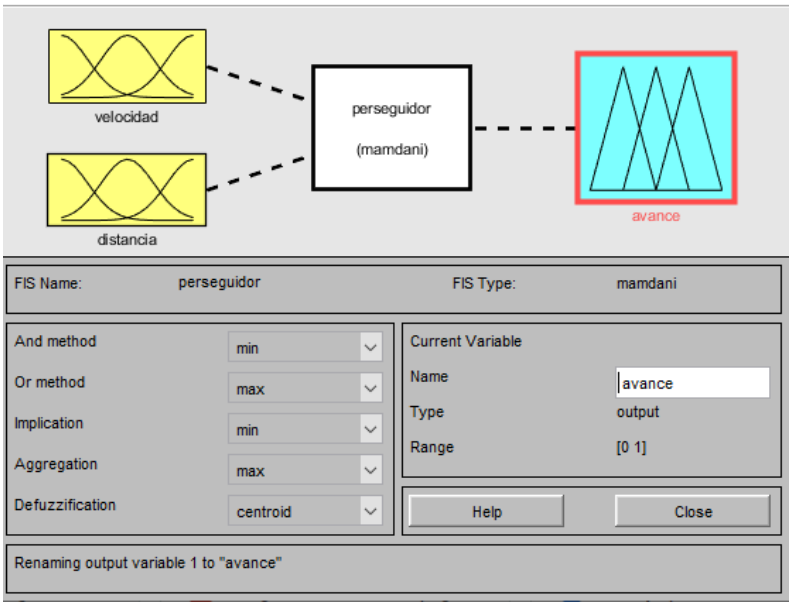

Figura A5 *- Vista previa del esquema para el controlador difuso, experiencia 2.*

Conjuntos difusos para la variable de velocidad

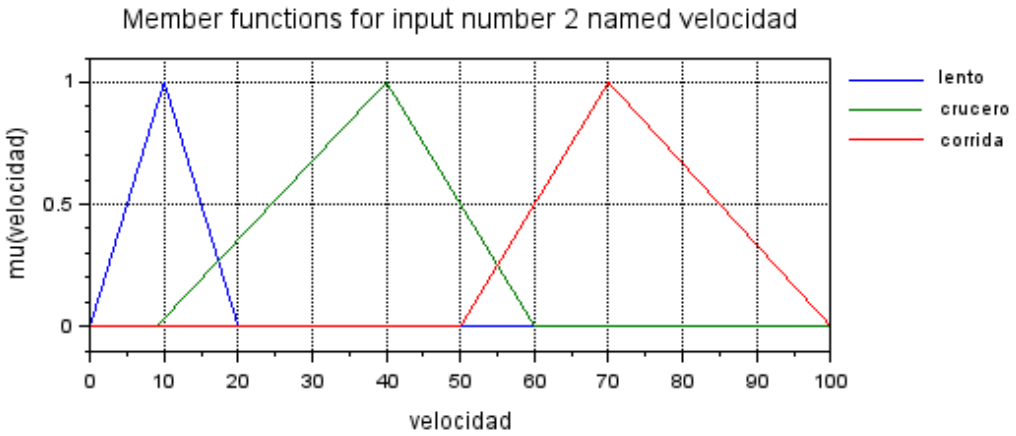

Figura A6 *- Conjuntos difusos, variable de velocidad.*

Conjuntos difusos para la variable de distancia

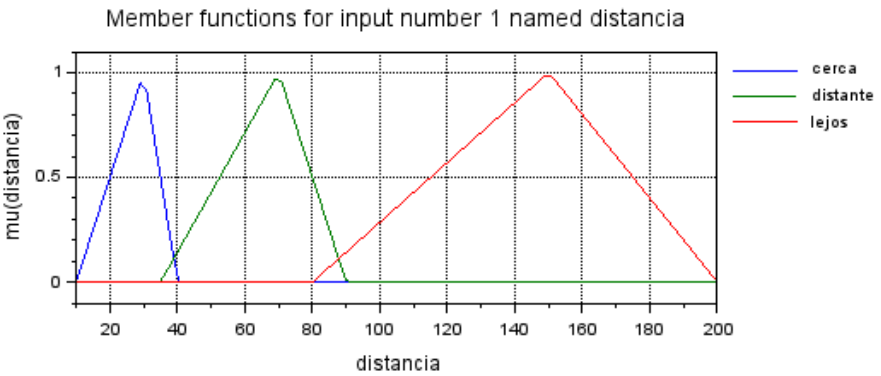

Figura A7 *- Conjuntos difusos, variable de distancia.*

Conjuntos para el cálculo de salida

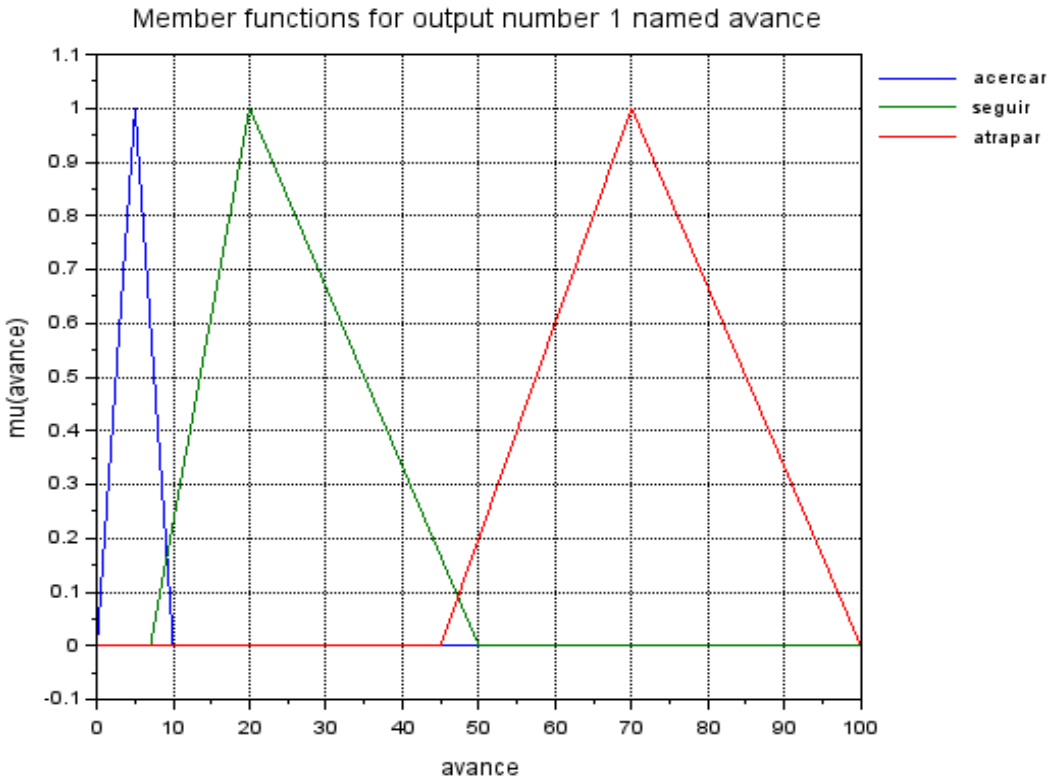

Figura A8 *- Conjuntos difusos, variable de salida.*

CÓDIGOS DE AMBAS EXPERIENCIAS

El código de color turquesa pertenece a la programación de componentes Arduino, mientras que el de color verde, es la programación del controlador difuso.

Código Fuente Experiencia 1 https://github.com/rosfix/ld_grp7_deteccion_estado_ambiental

```
//*********************************************************************
//*********************************************************************
#define FIS_TYPE float
#define FIS_RESOLUSION 101
#define FIS_MIN -3.4028235E+38
#define FIS_MAX 3.4028235E+38
typedef FIS_TYPE(*_FIS_MF)(FIS_TYPE, FIS_TYPE*);
typedef FIS_TYPE(*_FIS_ARR_OP)(FIS_TYPE, FIS_TYPE);
typedef FIS_TYPE(*_FIS_ARR)(FIS_TYPE*, int, _FIS_ARR_OP);
//*********************************************************************
//*********************************************************************

// Número de entradas al sistema de inferencia difusa
const int fis_gcI = 2;
// Número de salidas al sistema de inferencia difusa
const int fis_gcO = 1;
// Número de reglas para el sistema de inferencia difusa
const int fis_gcR = 16;

FIS_TYPE g_fisInput[fis_gcI];
FIS_TYPE g_fisOutput[fis_gcO];

//DECLARO EL SENSOR
float temperatura;
float humedad;
```

```
float salida;
#include <dht.h>
dht DHT;
#define DHT11_PIN 8

//DECLARO PINES DE LED RGB
int rojo=3;
int verde=4;
int azul=5;

//Interfaz Bluetooh
#include<SoftwareSerial.h>
SoftwareSerial BTSerial(2,3);

//Interfaz para Pantalla LCD
#include<Wire.h>
#include<LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2);
String menSalida;

// Setup
void setup()
{
   Serial.begin(9600);
   //INICIO EL LCD
   lcd.init();
   lcd.backlight();
   lcd.clear();
}
void loop()
{
      //LECTURA DEL SENSOR DHT11
      DHT.read11(DHT11_PIN);

      //TEMPERATURA//
      Serial.print("Temperatura = ");
      Serial.print(DHT.temperature);
      temperatura=DHT.temperature;
      Serial.println(" C");

     //HUMEDAD RELATIVA//
     Serial.print("Humedad = ");
     Serial.print(DHT.humidity);
     humedad=DHT.humidity;
     Serial.println(" %");
     delay(1000);

     if(temperatura>1 && temperatura<42 && humedad>1 && humedad<100){  //Dentro de los parametros del controlador difuso
         //CARGAR LAS VARIABLES DE ENTRADA AL CONTROLADOR DIFUSO
        // Read Input: temperatura
        g_fisInput[0] = temperatura;
        // Read Input: Humedad
        g_fisInput[1] = humedad;
        g_fisOutput[0] = 0;
        fis_evaluate();

        // SETEAR VALOR DE SALIDA EN SALIDA

        Serial.print("Salida: ");
        Serial.print(g_fisOutput[0]);
        salida=g_fisOutput[0];
        if(salida>=0 && salida<=0.4){
          if(salida>=0.3 && salida<=0.4){
           Serial.print(" - CLIMA ENTRE MALO Y NORMAL ");
           menSalida=" CLIMA:MyN";
           analogWrite(rojo,255);
           analogWrite(verde,255);
           analogWrite(azul,0);
          }else{
           Serial.print(" - CLIMA MALO ");
           menSalida=" CLIMA:M";
           analogWrite(rojo,255);
           analogWrite(verde,0);
           analogWrite(azul,0);
```

```
        }
      }
      if(salida>=0.3 && salida<=0.7){
        if(salida>=0.41 && salida<=0.5){
         Serial.print(" - CLIMA NORMAL ");
         menSalida=" CLIMA:N";
         analogWrite(rojo,0);
         analogWrite(verde,255);
         analogWrite(azul,0);
        }else{
          if(salida>=0.5 && salida<=0.7){
            Serial.print(" - CLIMA ENTRE NORMAL Y BUENO ");
            menSalida=" CLIMA:NyB";
            analogWrite(rojo,0);
            analogWrite(verde,255);
            analogWrite(azul,255);
          }
        }
      }
      if(salida>=0.5 && salida<=1){
        if(salida>=0.7 && salida<=1){
         Serial.print(" - CLIMA BUENO ");
         menSalida=" CLIMA:B";
         analogWrite(rojo,0);
         analogWrite(verde,0);
         analogWrite(azul,255);
        }
      }
      Serial.println("");
      delay(1000);
      //MOSTRAR EN EL LCD
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("T:");
      lcd.print(temperatura);
      lcd.print(" H:");
      lcd.print(humedad);
      lcd.setCursor(0,1);
      lcd.print("S:");
      lcd.print(salida);
      lcd.print(menSalida);
    }else{
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("FUERA DE RANGO");
    }

}

//*********************************************************************
// Funciones de soporte para el Sistema de Inferencia Difusa
//*********************************************************************
// Triangular Member Function
//FUNCION TRIANGULAR
FIS_TYPE fis_trimf(FIS_TYPE x, FIS_TYPE* p)
{
  FIS_TYPE a = p[0], b = p[1], c = p[2];
  FIS_TYPE t1 = (x - a) / (b - a);
  FIS_TYPE t2 = (c - x) / (c - b);
  if ((a == b) && (b == c)) return (FIS_TYPE) (x == a);
  if (a == b) return (FIS_TYPE) (t2*(b <= x)*(x <= c));
  if (b == c) return (FIS_TYPE) (t1*(a <= x)*(x <= b));
  t1 = min(t1, t2);
  return (FIS_TYPE) max(t1, 0);
}

//retorna minimos de la funciones de dependencia (Mamdani)
FIS_TYPE fis_min(FIS_TYPE a, FIS_TYPE b)
{
  return min(a, b);
}
//retorna maximos de la funciones de dependencia (Mamdani)
FIS_TYPE fis_max(FIS_TYPE a, FIS_TYPE b)
{
```

```
    return max(a, b);
}

FIS_TYPE fis_array_operation(FIS_TYPE *array, int size, _FIS_ARR_OP pfnOp)
{
    int i;
    FIS_TYPE ret = 0;
    if (size == 0) return ret;
    if (size == 1) return array[0];
    ret = array[0];
    for (i = 1; i < size; i++)
    {
        ret = (*pfnOp)(ret, array[i]);
    }
    return ret;
}

//*********************************************************************
// Datos para el sistema de inferencia difusa
//*********************************************************************
//Punteros a las implementaciones de funciones de membresia
_FIS_MF fis_gMF[] =
{
    fis_trimf
};

// Recuento de función miembro para cada entrada
int fis_gIMFCount[] = { 4, 4 };

// Recuento de función miembro para cada salida
int fis_gOMFCount[] = { 3 };

// Coeficientes para las funciones miembro de entrada
FIS_TYPE fis_gMFI0Coeff1[] = { -5, 5, 10 }; //SE CAMBIA A -5
FIS_TYPE fis_gMFI0Coeff2[] = { 8, 14, 18 };
FIS_TYPE fis_gMFI0Coeff3[] = { 15, 20, 25 };
FIS_TYPE fis_gMFI0Coeff4[] = { 20, 25, 50 };//SE CAMBIO 42 POR 50
FIS_TYPE* fis_gMFI0Coeff[] = { fis_gMFI0Coeff1, fis_gMFI0Coeff2, fis_gMFI0Coeff3, fis_gMFI0Coeff4 };
FIS_TYPE fis_gMFI1Coeff1[] = { 0, 20, 40 };
FIS_TYPE fis_gMFI1Coeff2[] = { 50, 55, 60 };
FIS_TYPE fis_gMFI1Coeff3[] = { 65, 85, 100 };
FIS_TYPE fis_gMFI1Coeff4[] = { 35, 55, 70 };
FIS_TYPE* fis_gMFI1Coeff[] = { fis_gMFI1Coeff1, fis_gMFI1Coeff2, fis_gMFI1Coeff3, fis_gMFI1Coeff4 };
FIS_TYPE** fis_gMFICoeff[] = { fis_gMFI0Coeff, fis_gMFI1Coeff };

// Coeficientes para las funciones miembro de salida
FIS_TYPE fis_gMFO0Coeff1[] = { 0, 0.2, 0.4167 }; //primer triangular de salida (MALO)
FIS_TYPE fis_gMFO0Coeff2[] = { 0.3, 0.5, 0.7 }; //segunda triagular de salida (NORMAL)
FIS_TYPE fis_gMFO0Coeff3[] = { 0.5, 0.8, 1 }; //tercera triangular de salida (BUENO)
FIS_TYPE* fis_gMFO0Coeff[] = { fis_gMFO0Coeff1, fis_gMFO0Coeff2, fis_gMFO0Coeff3 };
FIS_TYPE** fis_gMFOCoeff[] = { fis_gMFO0Coeff };

//Conjunto de funciones de membresía de entrada
int fis_gMFI0[] = { 0, 0, 0, 0 };
int fis_gMFI1[] = { 0, 0, 0, 0 };
int* fis_gMFI[] = { fis_gMFI0, fis_gMFI1};

// Output membership function set
//Conjunto de funciones de membresía de salida
int fis_gMFO0[] = { 0, 0, 0 };
int* fis_gMFO[] = { fis_gMFO0};


//pesos de reglas
FIS_TYPE fis_gRWeight[] = { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 };

// Tipo de regla
int fis_gRType[] = { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 };

// REGLAS DE ENTRADA
int fis_gRI0[] = { 1, 1 };
int fis_gRI1[] = { 1, 4 };
int fis_gRI2[] = { 1, 2 };
int fis_gRI3[] = { 1, 3 };
```

```
int fis_gRI4[] = { 2, 1 };
int fis_gRI5[] = { 2, 4 };
int fis_gRI6[] = { 2, 2 };
int fis_gRI7[] = { 2, 3 };
int fis_gRI8[] = { 3, 1 };
int fis_gRI9[] = { 3, 4 };
int fis_gRI10[] = { 3, 2 };
int fis_gRI11[] = { 3, 3 };
int fis_gRI12[] = { 4, 1 };
int fis_gRI13[] = { 4, 4 };
int fis_gRI14[] = { 4, 2 };
int fis_gRI15[] = { 4, 3 };
int* fis_gRI[] = { fis_gRI0, fis_gRI1, fis_gRI2, fis_gRI3, fis_gRI4, fis_gRI5, fis_gRI6, fis_gRI7, fis_gRI8, fis_gRI9, fis_gRI10,
fis_gRI11, fis_gRI12, fis_gRI13, fis_gRI14, fis_gRI15 };

// REGLAS DE SALIDA
int fis_gRO0[] = { 1 };
int fis_gRO1[] = { 2 };
int fis_gRO2[] = { 2 };
int fis_gRO3[] = { 1 };
int fis_gRO4[] = { 1 };
int fis_gRO5[] = { 2 };
int fis_gRO6[] = { 3 };
int fis_gRO7[] = { 1 };
int fis_gRO8[] = { 1 };
int fis_gRO9[] = { 3 };
int fis_gRO10[] = { 3 };
int fis_gRO11[] = { 1 };
int fis_gRO12[] = { 1 };
int fis_gRO13[] = { 3 };
int fis_gRO14[] = { 3 };
int fis_gRO15[] = { 1 };
int* fis_gRO[] = { fis_gRO0, fis_gRO1, fis_gRO2, fis_gRO3, fis_gRO4, fis_gRO5, fis_gRO6, fis_gRO7, fis_gRO8, fis_gRO9,
fis_gRO10, fis_gRO11, fis_gRO12, fis_gRO13, fis_gRO14, fis_gRO15 };

// Rango minimo de entrada
FIS_TYPE fis_gIMin[] = { 0, 0 };

// Rango maximo de entrada

FIS_TYPE fis_gIMax[] = { 50, 100 };//50 GRADOS Y 100 HUMEDAD

// Rango minimo de salida
FIS_TYPE fis_gOMin[] = { 0 };

// Rango maximo de salida
FIS_TYPE fis_gOMax[] = { 1 };

//*********************************************************************
// Funciones de soporte dependientes de los datos para el sistema de inferencia difusa
//*********************************************************************
FIS_TYPE fis_MF_out(FIS_TYPE** fuzzyRuleSet, FIS_TYPE x, int o)
{
  FIS_TYPE mfOut;
  int r;
  //MACHEAR LAS REGLAS CON LOS COEFICIENTES
  for (r = 0; r < fis_gcR; ++r)
  {
    int index = fis_gRO[r][o];
    if (index > 0)
    {
      index = index - 1;
      mfOut = (fis_gMF[fis_gMFO[o][index]])(x, fis_gMFOCoeff[o][index]);
    }
    else if (index < 0)
    {
      index = -index - 1;
      mfOut = 1 - (fis_gMF[fis_gMFO[o][index]])(x, fis_gMFOCoeff[o][index]);
    }
    else
    {
      mfOut = 0;
    }
```

```
        fuzzyRuleSet[0][r] = fis_min(mfOut, fuzzyRuleSet[1][r]);
    }
    return fis_array_operation(fuzzyRuleSet[0], fis_gcR, fis_max);
}

FIS_TYPE fis_defuzz_centroid(FIS_TYPE** fuzzyRuleSet, int o)
{
    FIS_TYPE step = (fis_gOMax[o] - fis_gOMin[o]) / (FIS_RESOLUSION - 1);
    FIS_TYPE area = 0;
    FIS_TYPE momentum = 0;
    FIS_TYPE dist, slice;
    int i;

    //calcular el área bajo la curva formada por las salidas MF
    for (i = 0; i < FIS_RESOLUSION; ++i){
        dist = fis_gOMin[o] + (step * i);
        slice = step * fis_MF_out(fuzzyRuleSet, dist, o);
        area += slice;
        momentum += slice*dist;
    }
    return ((area == 0) ? ((fis_gOMax[o] + fis_gOMin[o]) / 2) : (momentum / area));
}

//*********************************************************************
// Sistema de inferencia difusa
//*********************************************************************
void fis_evaluate()
{
    FIS_TYPE fuzzyInput0[] = { 0, 0, 0, 0 };
    FIS_TYPE fuzzyInput1[] = { 0, 0, 0, 0 };
    FIS_TYPE* fuzzyInput[fis_gcI] = { fuzzyInput0, fuzzyInput1, };
    FIS_TYPE fuzzyOutput0[] = { 0, 0, 0 };
    FIS_TYPE* fuzzyOutput[fis_gcO] = { fuzzyOutput0, };
    FIS_TYPE fuzzyRules[fis_gcR] = { 0 };
    FIS_TYPE fuzzyFires[fis_gcR] = { 0 };
    FIS_TYPE* fuzzyRuleSet[] = { fuzzyRules, fuzzyFires };
    FIS_TYPE sW = 0;

    // Transformación de entrada a entrada difusa
    int i, j, r, o;
    for (i = 0; i < fis_gcI; ++i)
    {
        for (j = 0; j < fis_gIMFCount[i]; ++j)
        {
            fuzzyInput[i][j] =
                (fis_gMF[fis_gMFI[i][j]])(g_fisInput[i], fis_gMFICoeff[i][j]);
        }
    }

    int index = 0;
    for (r = 0; r < fis_gcR; ++r)
    {
        if (fis_gRType[r] == 1)
        {
            fuzzyFires[r] = FIS_MAX;
            for (i = 0; i < fis_gcI; ++i)
            {
                index = fis_gRI[r][i];
                if (index > 0)
                    fuzzyFires[r] = fis_min(fuzzyFires[r], fuzzyInput[i][index - 1]); //MINIMOS DE FUNCIONES DE PERTENENCIA
                else if (index < 0)
                    fuzzyFires[r] = fis_min(fuzzyFires[r], 1 - fuzzyInput[i][-index - 1]);
                else
                    fuzzyFires[r] = fis_min(fuzzyFires[r], 1);
            }
        }
        else
        {
            fuzzyFires[r] = FIS_MIN;
            for (i = 0; i < fis_gcI; ++i)
            {
                index = fis_gRI[r][i];
                if (index > 0)
                    fuzzyFires[r] = fis_max(fuzzyFires[r], fuzzyInput[i][index - 1]);
```

```
            else if (index < 0)
                fuzzyFires[r] = fis_max(fuzzyFires[r], 1 - fuzzyInput[i][-index - 1]);
            else
                fuzzyFires[r] = fis_max(fuzzyFires[r], 0);
        }
    }

    fuzzyFires[r] = fis_gRWeight[r] * fuzzyFires[r];
    sW += fuzzyFires[r];
  }

  if (sW == 0)
  {
    for (o = 0; o < fis_gcO; ++o)
    {
      g_fisOutput[o] = ((fis_gOMax[o] + fis_gOMin[o]) / 2);
    }
  }
  else
  {
    for (o = 0; o < fis_gcO; ++o)
    {
      g_fisOutput[o] = fis_defuzz_centroid(fuzzyRuleSet, o); //SE ENVIA EL SETEO DE REGLAS A LA FUNCION CENTROIDE
    }
  }
}
```

Tabla A1 *- Código fuente de la experiencia 1.*

Código Fuente Experiencia 2:
https://github.com/rosfix/ld_seguidor_tpf/tree/main/fuzzy_tp_final_bis_sensor_motor

```
#include <Fuzzy.h>
#include <AFMotor.h>

// Fuzzy
Fuzzy *difuso = new Fuzzy();

// FuzzyInput: distancia
FuzzySet *cerca = new FuzzySet(0, 2, 50, 70);
FuzzySet *distante = new FuzzySet(90, 150, 250, 330);
FuzzySet *lejos = new FuzzySet(270, 320, 350, 350);

// FuzzyInput: velocidad
FuzzySet *lento = new FuzzySet(0, 5, 30, 35);
FuzzySet *crucero = new FuzzySet(25, 40, 60, 80);
FuzzySet *corrida = new FuzzySet(70, 85, 120, 150);

// FuzzyOutput: avance
FuzzySet *acercar = new FuzzySet(0, 2, 20, 50);
FuzzySet *seguir = new FuzzySet(25, 40, 60, 70);
FuzzySet *atrapar = new FuzzySet(55, 80, 100, 100);

const int EchoPin = 2;
const int TriggerPin = 13;

AF_Stepper motor1(256, 1);

AF_Stepper motor2(256, 2);

void setup() {

  // FuzzyInput
  FuzzyInput *distancia = new FuzzyInput(1);

  distancia->addFuzzySet(cerca);
  distancia->addFuzzySet(distante);
  distancia->addFuzzySet(lejos);
  difuso->addFuzzyInput(distancia);

  FuzzyInput *velocidad = new FuzzyInput(2);
```

```
 velocidad->addFuzzySet(lento);
 velocidad->addFuzzySet(crucero);
 velocidad->addFuzzySet(corrida);
 difuso->addFuzzyInput(velocidad);

// FuzzyOutput
 FuzzyOutput *avance = new FuzzyOutput(1);

 avance->addFuzzySet(acercar);
 avance->addFuzzySet(seguir);
 avance->addFuzzySet(atrapar);
 difuso->addFuzzyOutput(avance);

//R1: IF {distancia IS cerca} AND {velocidad IS lento} THEN {avance IS acercar} weigth=1.0
//R2: IF {distancia IS distante} AND {velocidad IS crucero} THEN {avance IS seguir} weigth=1.0
//R3: IF {distancia IS lejos} AND {velocidad IS crucero} THEN {avance IS atrapar} weigth=1.0

 FuzzyRuleAntecedent *cercaYlento = new FuzzyRuleAntecedent();
 cercaYlento->joinWithAND(cerca, lento);
 FuzzyRuleConsequent *acercamiento = new FuzzyRuleConsequent();
 acercamiento->addOutput(acercar);
 FuzzyRule *regla1 = new FuzzyRule(1, cercaYlento, acercamiento);
 difuso->addFuzzyRule(regla1);

 FuzzyRuleAntecedent *distanteYcrucero = new FuzzyRuleAntecedent();
 distanteYcrucero->joinWithAND(distante, crucero);
 FuzzyRuleConsequent *seguimiento = new FuzzyRuleConsequent();
 seguimiento->addOutput(acercar);
 FuzzyRule *regla2 = new FuzzyRule(2, distanteYcrucero, seguimiento);
 difuso->addFuzzyRule(regla2);

 FuzzyRuleAntecedent *lejosYcrucero = new FuzzyRuleAntecedent();
 lejosYcrucero->joinWithAND(lejos, crucero);
 FuzzyRuleConsequent *captura = new FuzzyRuleConsequent();
 captura->addOutput(atrapar);
 FuzzyRule *regla3 = new FuzzyRule(3, lejosYcrucero, captura);
 difuso->addFuzzyRule(regla3);

   FuzzyRuleAntecedent *cercaYcrucero = new FuzzyRuleAntecedent();
cercaYcrucero->joinWithAND(cerca, crucero);
 FuzzyRuleConsequent *cruzarcerca = new FuzzyRuleConsequent();
 cruzarcerca->addOutput(acercar);
 FuzzyRule *regla4 = new FuzzyRule(4, cercaYcrucero, cruzarcerca);
 difuso->addFuzzyRule(regla4);

   FuzzyRuleAntecedent *cercaYcorrida = new FuzzyRuleAntecedent();
 cercaYcorrida->joinWithAND(cerca, corrida);
 FuzzyRuleConsequent *correrCerca = new FuzzyRuleConsequent();
 correrCerca->addOutput(seguir);
 FuzzyRule *regla5 = new FuzzyRule(5, cercaYcorrida, correrCerca );
 difuso->addFuzzyRule(regla5);

   FuzzyRuleAntecedent *distanteYlento = new FuzzyRuleAntecedent();
 distanteYlento->joinWithAND(distante, lento);
 FuzzyRuleConsequent *sigilo = new FuzzyRuleConsequent();
 sigilo->addOutput(acercar);
 FuzzyRule *regla6 = new FuzzyRule(6, distanteYlento, sigilo);
 difuso->addFuzzyRule(regla6);

   FuzzyRuleAntecedent *distanteYcorrida = new FuzzyRuleAntecedent();
 distanteYcorrida->joinWithAND(distante, corrida);
 FuzzyRuleConsequent *sepierde = new FuzzyRuleConsequent();
 sepierde->addOutput(atrapar);
 FuzzyRule *regla7 = new FuzzyRule(7,distanteYcorrida, sepierde);
 difuso->addFuzzyRule(regla7);

   FuzzyRuleAntecedent *lejosYlento = new FuzzyRuleAntecedent();
 lejosYlento->joinWithAND(lejos, lento);
 FuzzyRuleConsequent *nomevio = new FuzzyRuleConsequent();
 nomevio->addOutput(seguir);
 FuzzyRule *regla8 = new FuzzyRule(8, lejosYlento, nomevio);
 difuso->addFuzzyRule(regla8);
```

```
    FuzzyRuleAntecedent *lejosYcorrida = new FuzzyRuleAntecedent();
  lejosYcorrida->joinWithAND(lejos, corrida);
  FuzzyRuleConsequent *seva = new FuzzyRuleConsequent();
  seva->addOutput(seguir);
  FuzzyRule *regla9 = new FuzzyRule(9, lejosYcorrida, seva);
  difuso->addFuzzyRule(regla9);

   Serial.begin(9600);
   pinMode(TriggerPin, OUTPUT);
   pinMode(EchoPin, INPUT);

   motor1.setSpeed(70); //rpm
   motor2.setSpeed(70); // rpm
}

int cm=0;
int cma=0;
int vel=0;

void loop() {
   velocidad();

  difuso->setInput(1, cm);
  difuso->setInput(2, vel);
  difuso->fuzzify();

  float avanceMotor = difuso->defuzzify(1);
   Serial.print(" Avance:");
   Serial.println(avanceMotor);


  int pasos=avanceMotor*3;
  Serial.print(" Pasos:");
  Serial.println(pasos);



 for (int paso = 0; paso <= pasos; paso++) {
   motor1.step(1, FORWARD, DOUBLE);
   motor2.step(1, BACKWARD, DOUBLE);
   }

}

void velocidad() {

   cm = ping(TriggerPin, EchoPin);
   vel=(cm-cma);

   Serial.print("D:");
   Serial.print(cm);
   Serial.print(" V:");
   Serial.println(vel);
   cma=cm;
   delay(1000);
}

int ping(int TriggerPin, int EchoPin) {
   long duration, distanceCm;

   digitalWrite(TriggerPin, LOW);  //para generar un pulso limpio ponemos a LOW 4us
   delayMicroseconds(4);
   digitalWrite(TriggerPin, HIGH);  //generamos Trigger (disparo) de 10us
   delayMicroseconds(10);
   digitalWrite(TriggerPin, LOW);

   duration = pulseIn(EchoPin, HIGH);  //medimos el tiempo entre pulsos, en microsegundos


   distanceCm = duration * 10 / 292/ 2;   //convertimos a distancia, en cm
   return distanceCm;
}
```

Tabla A2 *- Código fuente de la experiencia 2.*